

AZURE COSMOS DB CONSISTENCY LEVEL SELECTION STUDY

Project documentation • Problem statement • Architecture • Implementation and evaluation plan

Project Code	24CC3046-P048
Project Type	Database systems and distributed data consistency
Primary Platform	Microsoft Azure Cosmos DB
Team	3 teams • T048, T150, T252 • 12 students (as supplied)
Team Members	2400032107 2400032238 2400032136 2400032111

1. Problem Statement

Azure Cosmos DB is a globally distributed database that offers multiple consistency levels. Selecting a consistency level affects how quickly reads reflect writes, the latency experienced by applications, and the amount of coordination required across replicas. A level that is too weak for an access pattern can expose stale or out-of-order data and cause application bugs; a stronger level can increase latency and resource consumption. The project studies how to select an appropriate consistency level for three access patterns while balancing latency, cost and correctness.

2. Objectives

- Study the consistency guarantees provided by Azure Cosmos DB: Strong, Bounded Staleness, Session, Consistent Prefix and Eventual.
- Define three representative access patterns and their correctness requirements.
- Map each access pattern to a candidate consistency level and justify the selection.
- Compare observed latency and request-unit (RU) consumption under a controlled workload.
- Identify stale-read and ordering issues that may appear with weaker consistency.
- Document assumptions, experiment results, limitations and recommendations as evidence.

3. Scope and Assumptions

In scope: consistency-level concepts, a small Cosmos DB container, controlled read/write workloads, latency and RU observations, and comparison of outcomes.

Out of scope: production deployment, large-scale benchmarking, guaranteed cost forecasts, and use of real personal data. Use synthetic records. Actual availability of consistency options and configuration behavior should be verified for the selected Cosmos DB account/API and region.

Study assumption: the three access patterns below are proposed examples; replace them if your instructor has specified different patterns.

4. Use Cases / Access Patterns

Access pattern	Correctness need	Candidate level	Reason to test
A. User session reads	A user should see their own recent writes.	Session	Session token supports read-your-writes behavior for a client session.
B. Inventory / stock count	Reads must not exceed an agreed staleness window; strict freshness may be needed for critical updates.	Bounded Staleness or Strong	Compare bounded lag with stronger coordination and latency.
C. Activity feed / analytics	Temporary stale data is acceptable, but updates should not appear out of order.	Consistent Prefix	Evaluate ordered reads without requiring the latest write everywhere.

5. Required Technologies

- Microsoft Azure subscription and Azure portal — create and manage the database account.
- Azure Cosmos DB for NoSQL — database, container, partition key and consistency configuration.
- Azure Cosmos DB SDK (for example, .NET, Java, Python or JavaScript) — issue test reads and writes.
- Application Insights or simple timestamp logging — capture operation timing and errors.
- Azure Monitor / Cosmos DB metrics — inspect request units and throughput consumption.
- Excalidraw — create and export the architecture diagram.
- GitHub or a shared project folder — store source code, test data, screenshots and report.

6. Functional Requirements

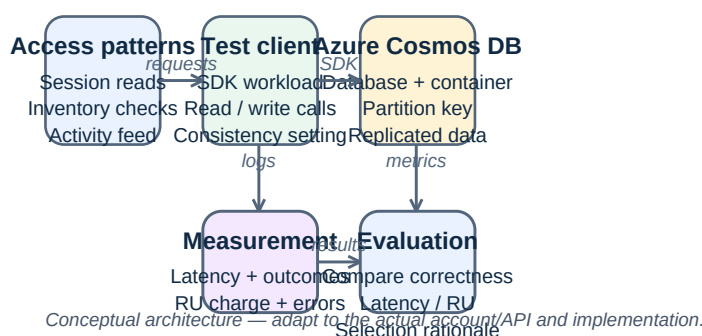
- Create a database and container with a documented partition key.
- Insert and update synthetic documents with version/timestamp fields.
- Run repeatable read/write workloads for each access pattern.
- Record operation type, consistency setting, latency, response and RU charge where available.
- Compare observed values across consistency settings and document stale or reordered reads.
- Present a traceable selection rationale for each access pattern.

7. Non-Functional Requirements

- Correctness: define expected behavior before running each test.
- Repeatability: keep workload size, region, data and test procedure consistent.
- Security: do not commit keys, connection strings or credentials.
- Cost control: use a small dataset and limited throughput; stop resources after testing.
- Observability: retain timestamps, RU charges, errors and screenshots.
- Clarity: distinguish measured results from expected behavior and assumptions.

8. System Architecture

The client workload generator sends reads and writes to Azure Cosmos DB. The account applies the selected consistency policy across replicas. The test harness records latency, responses and RU charges; results are then compared against each access pattern's correctness criteria.



9. Consistency-Level Selection Study

The study should treat the access pattern's correctness requirement as the starting point. Candidate levels are hypotheses to validate, not universal guarantees about application suitability. Use the same workload and deployment conditions when comparing measurements.

Level	High-level behavior	Study consideration
Strong	Reads provide the strongest consistency guarantee.	Use as a reference for correctness; observe latency and RU impact.
Bounded Staleness	Reads may lag, but within a configured bound.	Suitable to test when the application can define an acceptable lag.
Session	Provides session-scoped guarantees, including read-your-writes.	Test with the same client/session and with separate sessions.
Consistent Prefix	Reads do not observe writes out of order.	Test ordering when seeing the latest write immediately is not required.
Eventual	No ordering guarantee; replicas eventually converge.	Test tolerated staleness and convergence; unsuitable where stale data violates requirements.

10. Implementation and Experiment Process

Step 1 — Prepare the environment

Confirm Azure subscription access, select the Cosmos DB API and region, and review account-level consistency configuration.

Step 2 — Create the database

Create a Cosmos DB account or use an approved existing account. Create a database and container; document the partition key and throughput mode.

Step 3 — Prepare synthetic data

Create sample documents for session, inventory and feed scenarios. Include identifiers, values, versions and timestamps as appropriate.

Step 4 — Implement the test client

Use the official SDK to perform controlled reads and writes. Record the selected consistency setting and session context where applicable.

Step 5 — Define test cases

For each access pattern, write the expected result, the sequence of operations, the number of iterations and the acceptance criteria.

Step 6 — Run the experiments

Run each workload under the selected consistency levels, keeping region, dataset and operation sequence consistent. Repeat runs to reduce noise.

Step 7 — Capture measurements

Record end-to-end latency, RU charge, success/failure, returned version and any stale or out-of-order observations.

Step 8 — Analyze and select

Compare results with the stated correctness needs. Explain trade-offs and choose a level per access pattern only where evidence supports it.

Step 9 — Document evidence

Save architecture, configuration screenshots, test code, anonymized logs, result tables and limitations.

Step 10 — Demonstrate

Explain the problem, architecture, test method, measured results, selection rationale, bottlenecks and future improvements.

11. Evaluation Metrics and Result Template

Record actual measurements during implementation. Do not fill this table with assumed values. Run multiple iterations and report the sample count and summary statistic used (for example, median and p95 latency).

Pattern	Consistency	Median latency	p95 latency	RU/op	Correctness notes
Session reads	Strong	—	—	—	To be measured
Session reads	Bounded Staleness	—	—	—	To be measured
Session reads	Session	—	—	—	To be measured
Session reads	Consistent Prefix	—	—	—	To be measured
Session reads	Eventual	—	—	—	To be measured
Inventory checks	Strong	—	—	—	To be measured
Inventory checks	Bounded Staleness	—	—	—	To be measured
Inventory checks	Session	—	—	—	To be measured
Inventory checks	Consistent Prefix	—	—	—	To be measured
Inventory checks	Eventual	—	—	—	To be measured
Activity feed	Strong	—	—	—	To be measured
Activity feed	Bounded Staleness	—	—	—	To be measured
Activity feed	Session	—	—	—	To be measured
Activity feed	Consistent Prefix	—	—	—	To be measured
Activity feed	Eventual	—	—	—	To be measured

12. Bottlenecks Students May Encounter

- Weaker consistency can introduce stale-read or ordering bugs that are discovered late; define correctness tests before implementation.
- Request Unit (RU) provisioning and consumption can drive unexpected cost; begin with a small workload, monitor usage and stop resources when finished.
- Latency varies with region, network, partitioning, SDK retries and workload shape; record these conditions and avoid overgeneralizing results.
- Session consistency tests can be misleading if session context is not preserved; document client/session handling.
- Concurrent writes can make results nondeterministic; use controlled operation sequences and include timestamps or version numbers.
- Account-level consistency configuration and SDK behavior may constrain what can be tested; verify the actual environment before claiming support.

13. Suggested Repository Structure

```
Azure-CosmosDB-Consistency-Study/  
■■■ README.md  
■■■ docs/  
■ ■■■ problem-statement.md  
■ ■■■ architecture.excalidraw  
■ ■■■ experiment-method.md  
■ ■■■ results.md
```

- src/
- sample-data/
- results/
- screenshots/
- .gitignore

14. Expected Outcome

A documented and repeatable comparison of consistency behavior for three access patterns, supported by an architecture diagram, test implementation, measured latency and RU observations, correctness evidence and a reasoned selection rationale. Actual conclusions depend on the experiments performed.

15. Conclusion

The project frames consistency selection as a trade-off between application correctness, latency and resource consumption. By defining access-pattern requirements first and validating candidate levels through controlled tests, the team can document where weaker consistency is acceptable and where stronger guarantees may be required. The final report should distinguish measured findings from assumptions and limitations.